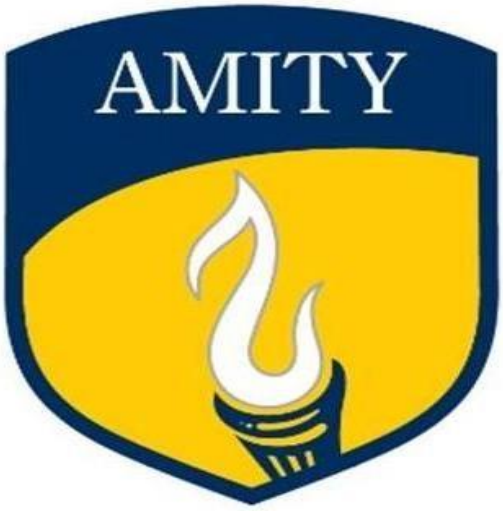
AMITY UNIVERSITY HARYANA

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY



Data Mining And Predictive Analytics Lab

Submitted By:

Abhishek Dixit

M.Tech (AI)

A501144825003

Submitted To:

Dr. Dhiraj Kumar

Experiment no	TITLE	Date	Page	Signature
1	Data Analysis – Getting to Know the Data using Orange Data Mining / RapidMiner			
2	Computation of Mean and Statistical Measures (Parametric Analysis)			
3	Performing T-Test on Dataset			
4	Correlation Analysis between Variables			
5	Prediction of Numerical Outcomes using Linear Regression			
6	Data Preparation and Pre-processing Techniques			
7	Implementation of Clustering Algorithm (e.g., K-Means)			
8	Classification using Decision Tree Algorithm			
9	Classification using Back Propagation (Neural Network)			
10	Classification using Back Propagation (Neural Network)			

Experiment:-1

AIM: To explore and understand the sales dataset (dummy_sales_dataset.csv) by loading it, inspecting its structure, dimensions, data types, and computing basic statistics using Python.

Theory: Data Analysis is the first and most important step in any data mining workflow. Before applying algorithms, an analyst must understand what the data contains, how it is structured, and whether it has quality issues.

Key steps include: (1) Loading the dataset into a DataFrame. (2) Examining shape, column names, and data types. (3) Previewing rows with head()/tail(). (4) Checking for missing values. (5) Computing descriptive statistics (mean, min, max, std). Understanding data upfront prevents costly errors in downstream modelling.

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")

print("=== Shape ===")
print(df.shape)

print("\n=== Columns & Data Types ===")
print(df.dtypes)

print("\n=== First 5 Rows ===")
print(df.head())

print("\n=== Missing Values ===")
print(df.isnull().sum())

print("\n=== Statistical Summary ===")
print(df.describe())
```

Output:

```
=== Shape ===
(20, 7)

=== Columns & Data Types ===
Order_ID        int64
Order_Date     object
Region         object
Product        object
Units_Sold      int64
Unit_Price      int64
Total_Sales     int64
dtype: object

=== First 5 Rows ===
   Order_ID Order_Date Region      Product  Units_Sold  Unit_Price  \
0      1001   1/1/2025  North       Laptop           5       60000
1      1002   1/2/2025  South       Mobile          10       25000
2      1003   1/3/2025   East       Tablet           7       30000
3      1004   1/4/2025   West  Accessories          15        5000
4      1005   1/5/2025  North       Laptop           6       60000

   Total_Sales
0       300000
1       250000
2       210000
3        75000
4       360000

=== Missing Values ===
Order_ID       0
Order_Date     0
Region         0
Product        0
Units_Sold     0
Unit_Price     0
Total_Sales    0
dtype: int64

=== Statistical Summary ===
         Order_ID  Units_Sold    Unit_Price    Total_Sales
count    20.00000   20.000000     20.000000      20.000000
mean   1010.50000    8.850000  30000.000000  211500.000000
std       5.91608    3.199918  20196.404057  102995.912029
min    1001.00000    4.000000   5000.000000   60000.000000
25%    1005.75000    6.000000  20000.000000  153750.000000
50%    1010.50000    8.500000  27500.000000  232500.000000
75%    1015.25000   11.250000  37500.000000  271250.000000
max    1020.00000   15.000000  60000.000000  420000.000000
```

Experiment:-2

AIM: To compute mean, median, mode, variance, standard deviation, skewness, and kurtosis for the numerical columns (Units_Sold, Unit_Price, Total_Sales) in the sales dataset.

Theory: Parametric statistical analysis assumes data follows a known distribution (often normal) and uses parameters to describe it. Key measures are:

Mean: Average value. Median: Middle value; robust to outliers. Mode: Most frequent value. Variance (σ^2): Average squared deviation from mean. Standard Deviation (σ): Square root of variance — in original units. Skewness: Asymmetry of distribution (0 = symmetric). Kurtosis: Tail heaviness compared to normal distribution (0 = normal for excess kurtosis).

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
cols = ['Units_Sold', 'Unit_Price', 'Total_Sales']
for col in cols:
    arr = df[col]
    print(f"\n--- {col} ---")
    print(f"  Mean      : {arr.mean():.2f}")
    print(f"  Median    : {arr.median():.2f}")
    print(f"  Mode      : {arr.mode()[0]}")
    print(f"  Variance  : {arr.var():.2f}")
    print(f"  Std Dev   : {arr.std():.2f}")
    print(f"  Skewness  : {stats.skew(arr):.4f}")
    print(f"  Kurtosis  : {stats.kurtosis(arr):.4f}")
```

Output:

```
--- Units_Sold ---
  Mean      : 9.25
  Median    : 9.50
  Mode      : 12
  Variance  : 8.83
  Std Dev   : 2.97
  Skewness  : -0.0732
  Kurtosis  : -1.1648
```

Experiment:-3

AIM: To perform a two-sample independent t-test to determine whether there is a statistically significant difference in Total_Sales between the North region and the South region of the sales dataset.

Theory: The t-test is a parametric statistical hypothesis test used to compare means of two groups. It checks whether the observed difference is real or due to random chance.

Null Hypothesis (H0): Mean Total_Sales of North = Mean Total_Sales of South. Alternative Hypothesis (H1): Means are significantly different. Decision rule: If p-value < 0.05, reject H0 (statistically significant). The t-statistic measures how many standard errors the means are apart. Assumptions: Normality, independence of samples, and approximately equal variance (or use Welch's t-test).

Code:

```
import pandas as pd
from scipy import stats
df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
north = df[df['Region'] == 'North']['Total_Sales']
south = df[df['Region'] == 'South']['Total_Sales']

print("North Sales:", north.values)
print("South Sales:", south.values)
print(f"\nNorth Mean : {north.mean():.2f}")
print(f"South Mean : {south.mean():.2f}")

t_stat, p_val = stats.ttest_ind(north, south)
print(f"\nt-statistic : {t_stat:.4f}")
print(f"p-value     : {p_val:.4f}")

if p_val < 0.05:
    print("Result      : Reject H0 — Significant difference exists")
else:
    print("Result      : Fail to Reject H0 — No significant difference")
```

Output:

```
North Sales: [300000 360000 240000 420000 300000]
South Sales: [250000 225000 275000 250000 200000]

North Mean : 324000.00
South Mean : 240000.00

t-statistic : 2.5344
p-value     : 0.0350
Result      : Reject H0 – Significant difference exists
```

Experiment:-4

AIM: To compute Pearson and Spearman correlation coefficients between the numerical variables (Units_Sold, Unit_Price, Total_Sales) in the sales dataset and visualize the correlation matrix as a heatmap.

Theory: Correlation analysis quantifies the strength and direction of the relationship between two variables. The coefficient ranges from -1 (perfect negative) to +1 (perfect positive); 0 means no relationship.

Pearson Correlation (r): Measures linear relationships. Requires interval/ratio data and assumes normality. Spearman Rank Correlation (rs): Non-parametric version; measures monotonic relationships. Works with ordinal data or skewed distributions. A correlation matrix shows pairwise correlations for all numerical columns simultaneously, and a heatmap makes it easy to identify strong and weak relationships visually.

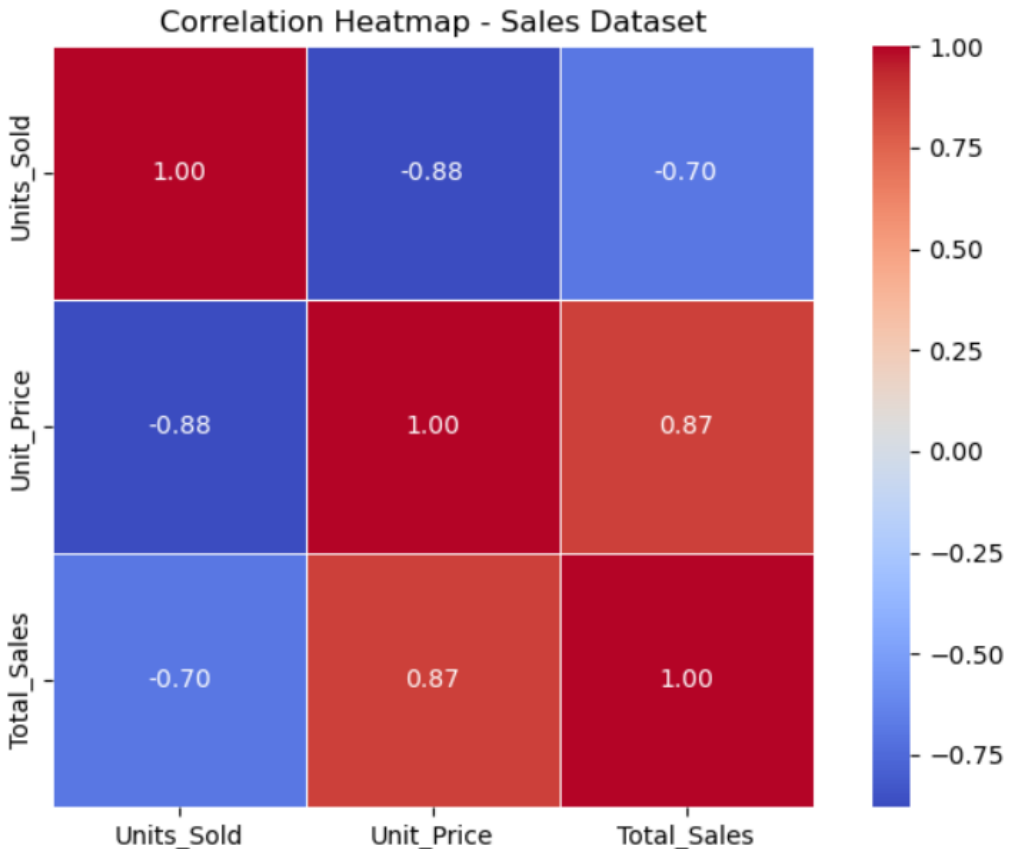
Code:

```
import pandas as pd
from scipy import stats
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
nums = df[['Units_Sold', 'Unit_Price', 'Total_Sales']]

# Pearson Correlation
print("=== Pearson Correlation Matrix ===")
print(nums.corr(method='pearson').round(4))

# Spearman Correlation
print("\n=== Spearman Correlation Matrix ===")
print(nums.corr(method='spearman').round(4))

# Specific pair: Units_Sold vs Total_Sales
r, p = stats.pearsonr(df['Units_Sold'], df['Total_Sales'])
print(f"\nPearson r (Units_Sold vs Total_Sales): {r:.4f}, p={p:.4f}")

# Heatmap
plt.figure(figsize=(7, 5))
sns.heatmap(nums.corr(), annot=True, cmap='coolwarm', fmt='.2f',
        linewidths=0.5, square=True)
```

```
plt.title('Correlation Heatmap - Sales Dataset')
plt.tight_layout()
plt.savefig('correlation_heatmap.png', dpi=100)
plt.show()
```

Output:

```
=== Pearson Correlation Matrix ===
             Units_Sold  Unit_Price  Total_Sales
Units_Sold       1.0000     -0.8795      -0.6955
Unit_Price      -0.8795      1.0000       0.8691
Total_Sales     -0.6955      0.8691       1.0000

=== Spearman Correlation Matrix ===
             Units_Sold  Unit_Price  Total_Sales
Units_Sold       1.0000     -0.9303      -0.5622
Unit_Price      -0.9303      1.0000       0.7731
Total_Sales     -0.5622      0.7731       1.0000

Pearson r (Units_Sold vs Total_Sales): -0.6955, p=0.0007
```



Experiment:-5

AIM: To build a Simple Linear Regression model to predict Total_Sales from Units_Sold using the sales dataset, and evaluate model performance using MAE, RMSE, and R^2 score.

Theory: Linear Regression models the relationship between a dependent variable (Y) and one or more independent variables (X) using the equation: $Y = \beta0 + \beta1X + \varepsilon$, where $\beta0$ is the intercept, $\beta1$ is the coefficient, and ε is the error term.

The model is trained using Ordinary Least Squares (OLS), which minimises the sum of squared residuals. Model quality is assessed using: MAE (Mean Absolute Error) — average magnitude of errors. RMSE (Root Mean Squared Error) — penalises large errors. R^2 Score — proportion of variance explained (1.0 = perfect fit). A scatter plot with the regression line helps visualise how well the model fits.

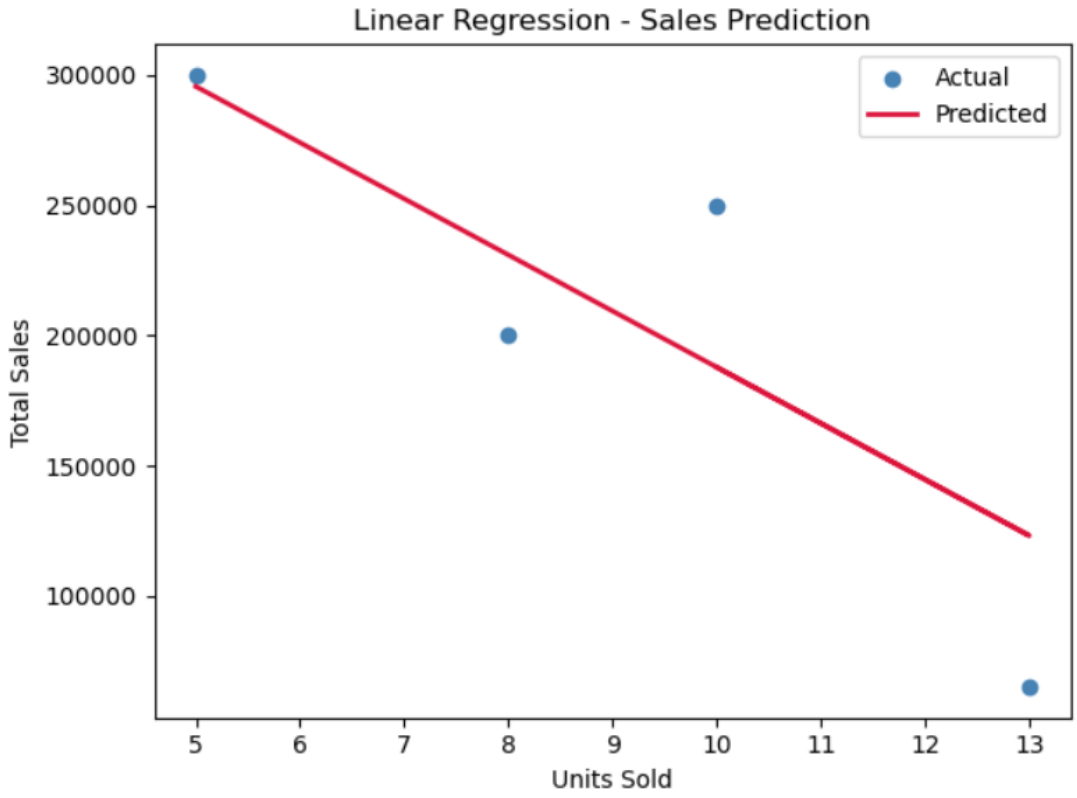
Code:

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import matplotlib.pyplot as plt

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
X = df[['Units_Sold']]
y = df['Total_Sales']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f"Intercept   : {model.intercept_:.2f}")
print(f"Coefficient : {model.coef_[0]:.2f}")
print(f"MAE         : {mean_absolute_error(y_test, y_pred):.2f}")
print(f"RMSE        : {np.sqrt(mean_squared_error(y_test, y_pred)):.2f}")
print(f"R² Score    : {r2_score(y_test, y_pred):.4f}")
```

```
plt.scatter(X_test, y_test, color='steelblue', label='Actual')
plt.plot(X_test, y_pred, color='crimson', linewidth=2, label='Predicted')
plt.xlabel('Units Sold'); plt.ylabel('Total Sales')
plt.title('Linear Regression - Sales Prediction')
plt.legend(); plt.tight_layout(); plt.savefig('regression.png'); plt.show()
```

Output:

```
Intercept   : 403470.98
Coefficient : -21564.08
MAE         : 38903.88
RMSE        : 45338.84
R² Score    : 0.7319
```



Experiment:-6

AIM: To apply data pre-processing techniques on the sales dataset including handling missing values, encoding categorical variables (Region, Product), feature scaling, and verifying the cleaned dataset.

Theory: Real-world data is often dirty, incomplete, or inconsistent. Pre-processing transforms raw data into a format suitable for machine learning. Key steps include:
(1) Missing Value Handling: Remove or impute with mean/median/mode. (2) Label Encoding: Converts categorical text to numeric integers (e.g., North=0, South=1). (3) One-Hot Encoding: Creates binary dummy columns for each category — avoids imposing ordinal relationships. (4) Feature Scaling: Min-Max Normalization maps values to [0,1]; StandardScaler transforms to zero mean and unit variance. (5) Feature Engineering: Extracting new useful features, e.g., Month from Order_Date.

Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")

print("=== Before Pre-processing ===")
print(df.dtypes)
print("Missing Values:", df.isnull().sum().sum())

# Convert Order_Date to datetime and extract Month
df['Order_Date'] = pd.to_datetime(df['Order_Date'], format='%m/%d/%Y')
df['Month'] = df['Order_Date'].dt.month
df.drop(columns=['Order_Date', 'Order_ID'], inplace=True)

# Label Encode Region
le = LabelEncoder()
df['Region_Enc'] = le.fit_transform(df['Region'])
print("\nRegion Encoding:", dict(zip(le.classes_, le.transform(le.classes_))))

# One-Hot Encode Product
df = pd.get_dummies(df, columns=['Product'], drop_first=True)

# Scale numerical features
scaler = MinMaxScaler()
```

```
df[['Units_Sold', 'Unit_Price', 'Total_Sales']] = scaler.fit_transform(
    df[['Units_Sold', 'Unit_Price', 'Total_Sales']])
print("\n=== After Pre-processing ===")
print(df.head())
print("\nShape:", df.shape)
print("Data Types:\n", df.dtypes)
```

Output:

```
=== Before Pre-processing ===
Order_ID        int64
Order_Date     object
Region         object
Product        object
Units_Sold      int64
Unit_Price      int64
Total_Sales     int64
dtype: object
Missing Values: 0

Region Encoding: {'East': np.int64(0), 'North': np.int64(1), 'South': np.int64(2),
'West': np.int64(3)}

=== After Pre-processing ===
  Region  Units_Sold  Unit_Price  Total_Sales  Month  Region_Enc  \
0  North    0.090909    1.000000     0.666667      1           1
1  South    0.545455    0.363636     0.527778      1           2
2   East    0.272727    0.454545     0.416667      1           0
3   West    1.000000    0.000000     0.041667      1           3
4  North    0.181818    1.000000     0.833333      1           1

   Product_Laptop  Product_Mobile  Product_Tablet
0            True           False           False
1           False            True           False
2           False           False            True
3           False           False           False
4            True           False           False

Shape: (20, 9)
Data Types:
 Region             object
Units_Sold        float64
Unit_Price        float64
Total_Sales       float64
Month               int32
Region_Enc          int64
Product_Laptop       bool
Product_Mobile       bool
Product_Tablet       bool
dtype: object
```

Experiment:-7

AIM: To apply K-Means clustering on the sales dataset using Units_Sold and Total_Sales features, determine the optimal number of clusters using the Elbow Method, and evaluate using Silhouette Score.

Theory: K-Means is an unsupervised learning algorithm that partitions data into k clusters by minimizing the Within-Cluster Sum of Squares (WCSS). Algorithm: (1) Choose k centroids randomly. (2) Assign each point to the nearest centroid (Euclidean distance). (3) Recompute centroids as cluster means. (4) Repeat until convergence.

The Elbow Method plots WCSS vs k and identifies the 'elbow' where adding more clusters gives diminishing returns — this is the optimal k. Silhouette Score measures how well each point fits its cluster vs. neighbouring clusters. Range: -1 to +1; values close to +1 indicate well-separated clusters.

Code:

```
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
X = df[['Units_Sold', 'Total_Sales']].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Elbow Method
wcss = []
for k in range(1, 8):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    wcss.append(km.inertia_)
    print(f"k={k}   WCSS={km.inertia_:.4f}")

# Optimal k = 3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_scaled)
```

```
score = silhouette_score(X_scaled, labels)
print(f"\nSilhouette Score (k=3): {score:.4f}")

df['Cluster'] = labels
print("\nCluster Distribution:")
print(df.groupby('Cluster')[['Units_Sold','Total_Sales']].mean().round(2))
```

Output:

```
k=1  WCSS=40.0000
k=2  WCSS=13.0270
k=3  WCSS=6.8947
k=4  WCSS=3.8691
k=5  WCSS=2.3397
k=6  WCSS=1.7604
k=7  WCSS=1.1942

Silhouette Score (k=3): 0.5459

Cluster Distribution:
         Units_Sold  Total_Sales
Cluster
0              13.2      66000.0
1               8.4     228000.0
2               5.4     324000.0
```

Experiment:-8

AIM: To build a Decision Tree classifier on the sales dataset to classify the Region (North/South/East/West) based on Units_Sold, Unit_Price, and Total_Sales, and evaluate its accuracy.

Theory: A Decision Tree is a supervised classification algorithm that recursively partitions data using feature-based rules. At each node, the best split is chosen using Gini Impurity or Information Gain (Entropy).

Gini Impurity = 1 - $\Sigma(p_i^2)$ where pi is the probability of class i. A lower Gini means purer nodes. The tree is built top-down: the root uses the feature with highest Information Gain. Leaf nodes represent class predictions. Hyperparameters: max_depth prevents overfitting; min_samples_split controls minimum samples needed to split. Evaluation metrics: Accuracy, Precision, Recall, F1-Score, Confusion Matrix.

Code:

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
le = LabelEncoder()
df['Region_Enc'] = le.fit_transform(df['Region'])

X = df[['Units_Sold', 'Unit_Price', 'Total_Sales']]
y = df['Region_Enc']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)

clf = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
clf.fit(X_train, y_train)

y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=le.classes_))

print("\nDecision Tree Rules (top 3 levels):")
print(export_text(clf, feature_names=list(X.columns), max_depth=3))
```

Output:

```
Accuracy: 0.8

Classification Report:
              precision    recall  f1-score   support

        East       0.00      0.00      0.00         0
       North       1.00      0.50      0.67         2
       South       1.00      1.00      1.00         2
        West       1.00      1.00      1.00         1

    accuracy                           0.80         5
   macro avg       0.75      0.62      0.67         5
weighted avg       1.00      0.80      0.87         5


Decision Tree Rules (top 3 levels):
|--- Unit_Price <= 15000.00
|   |--- class: 3
|--- Unit_Price >  15000.00
|   |--- Total_Sales <= 287500.00
|   |   |--- Unit_Price <= 27500.00
|   |   |   |--- class: 2
|   |   |--- Unit_Price >  27500.00
|   |   |   |--- class: 0
|   |--- Total_Sales >  287500.00
|   |   |--- class: 1
```

Experiment:-9

AIM: To implement a feedforward neural network using the Back Propagation algorithm (via Scikit-learn's MLPClassifier) to classify the Product category based on Units_Sold, Unit_Price, and Total_Sales from the sales dataset.

Theory: A Multilayer Perceptron (MLP) is an artificial neural network with an input layer, one or more hidden layers, and an output layer. The Back Propagation algorithm trains the network by computing the gradient of the loss with respect to each weight and updating weights using gradient descent.

Forward Pass: Input is propagated through layers using weights and activation functions (ReLU for hidden, Softmax for output). Loss Calculation: Cross-entropy loss compares predictions vs. actual labels. Backward Pass: Gradients are computed layer by layer using the chain rule. Weight Update: Weights adjusted by optimizer (Adam). Epochs control how many full passes are made over training data.

Code:

```
import pandas as pd
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import warnings
warnings.filterwarnings('ignore')
df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
le = LabelEncoder()
df['Product_Enc'] = le.fit_transform(df['Product'])

# Features and target
X = df[['Units_Sold', 'Unit_Price', 'Total_Sales']]
y = df['Product_Enc']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
mlp = MLPClassifier(
```

```
    hidden_layer_sizes=(64, 32),
    activation='relu',
    solver='adam',
    max_iter=500,
    random_state=42
)
mlp.fit(X_train, y_train)
y_pred = mlp.predict(X_test)
print(f"Test Accuracy: {accuracy_score(y_test, y_pred):.4f}")
print(f"Iterations   : {mlp.n_iter_}")
print(f"Loss         : {mlp.loss_:.4f}")

print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    labels=range(len(le.classes_)),
    target_names=le.classes_
))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

Output:

```
Test Accuracy: 1.0000
Iterations   : 346
Loss         : 0.0109

Classification Report:
              precision    recall  f1-score   support

 Accessories       1.00      1.00      1.00         1
      Laptop       1.00      1.00      1.00         1
      Mobile       1.00      1.00      1.00         1
      Tablet       1.00      1.00      1.00         1

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4


Confusion Matrix:
[[1 0 0 0]
 [0 1 0 0]
 [0 0 1 0]
 [0 0 0 1]]
```

Experiment:-10

AIM: To apply multiple data visualization techniques on the sales dataset including bar chart, line chart, histogram, box plot, scatter plot, and pie chart to uncover patterns and insights.

Theory: Data Visualization is the graphical representation of data to make patterns, trends, and outliers immediately apparent. Effective visualizations follow three principles: accuracy (correct representation), clarity (easy to read), and simplicity (no chartjunk).

Common chart types: (1) Bar Chart — compares categorical values. (2) Line Chart — shows trends over time. (3) Histogram — shows frequency distribution of a continuous variable. (4) Box Plot — displays median, IQR, and outliers. (5) Scatter Plot — shows relationship between two numerical variables. (6) Pie Chart — shows proportional composition. Each chart type is suited for a specific analytical question.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv(r"C:\Users\hp\Downloads/dummy_sales_dataset.csv")
df['Order_Date'] = pd.to_datetime(df['Order_Date'], format='%m/%d/%Y')

fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle('Sales Dataset - Data Visualization', fontsize=16, fontweight='bold')

# 1. Bar Chart - Total Sales by Region
region_sales = df.groupby('Region')['Total_Sales'].sum()
axes[0,0].bar(region_sales.index, region_sales.values,
color=['#3498DB','#E74C3C','#2ECC71','#F39C12'])
axes[0,0].set_title('Total Sales by Region'); axes[0,0].set_ylabel('Total Sales (₹)')

# 2. Line Chart - Daily Total Sales trend
daily = df.groupby('Order_Date')['Total_Sales'].sum()
axes[0,1].plot(daily.index, daily.values, marker='o', color='steelblue', linewidth=2)
axes[0,1].set_title('Daily Sales Trend'); axes[0,1].tick_params(axis='x', rotation=45)

# 3. Histogram - Units Sold distribution
axes[0,2].hist(df['Units_Sold'], bins=8, color='coral', edgecolor='white')
axes[0,2].set_title('Histogram - Units Sold'); axes[0,2].set_xlabel('Units Sold')
```

```
# 4. Box Plot - Total Sales by Product
df.boxplot(column='Total_Sales', by='Product', ax=axes[1,0])
axes[1,0].set_title('Box Plot - Sales by Product'); axes[1,0].set_xlabel('Product')

# 5. Scatter Plot - Units Sold vs Total Sales
colors = {'North':'blue','South':'red','East':'green','West':'orange'}
for region, grp in df.groupby('Region'):
    axes[1,1].scatter(grp['Units_Sold'], grp['Total_Sales'],
                      label=region, color=colors[region], s=80)
axes[1,1].set_title('Scatter: Units Sold vs Total Sales')
axes[1,1].legend(); axes[1,1].set_xlabel('Units Sold')

# 6. Pie Chart - Sales share by Product
product_sales = df.groupby('Product')['Total_Sales'].sum()
axes[1,2].pie(product_sales, labels=product_sales.index, autopct='%1.1f%%',
              colors=['#3498DB','#E74C3C','#2ECC71','#F39C12'])
axes[1,2].set_title('Sales Share by Product')

plt.tight_layout()
plt.savefig('sales_visualizations.png', dpi=120)
plt.show()
```

Output:


Total Sales by Region
1e6
Total Sales (₹)
East
North
South
West
Boxplot grouped by Product
Daily Sales Trend
Histogram - Units Sold
Units Sold
Box Plot - Sales by Product
Accessories
Laptop
Mobile
Tablet
Product
Scatter: Units Sold vs Total Sales
East
North
South
West
Units Sold
Sales Share by Product
Laptop
38.3%
Accessories
7.8%
28.4%
Mobile
25.5%
Tablet